校园时间管家 CampusTime

技术文档（AI 创新应用挑战赛 · 个人赛提交版）

判定层 / 执行层分离 · Propose/Execute 确定性架构 · AI 协同细粒度开发

项目	CampusTime / 校园时间管家 Agent
定位	校园 AI 服务助手（学生活端 / 驿站商户侧 / 管理控制台）
技术栈	Python + FastAPI + 原生 HTML/JS + 自写轻量 ReAct Agent + DeepSeek
仓库	github.com/zhouyuliang0606/campus-time-agent（Public）
规模	77 个 API + 7 个页面（共 84 条路由） / 1056 个细粒度 commit，横跨 39 个开发日（2026-09-23 → 11-04） / 189 份 Prompt 快照 + 42 份施工单原件
自检	tests/run_all.py：通过 6039 项，失败 0 项（111 批），2026-11-04 实测
文档版本	2026-11-04

摘要

校园时间管家是一个把分散的校园信息（课表、待办、快递外卖、知识问答）收口到同一个 AI 助手的项目。它有两条互锁的地基：

- Propose / Execute 分离：Agent 只产出「提案」（read-only 工具），学生没在确认卡上点确认之前，系统一个字节都不写库；确认卡的显示由系统盖章的 kind 标记驱动，而非模型自觉。
- 判定层 / 执行层分离：把「听懂人话」和「动手执行」彻底切开。判定层（样本库 → 语义层 → 关键词兜底）负责把一句话听懂并归类，可以换、可以兜底、可以交给大模型；执行层（查空档、排期、出卡、写库）由系统独占，模型永远碰不到写入接口。

本文档说明系统架构与三条端到端工作流、这两条核心设计的成因与落地、2026-09-26 起至 11-04 的六次架构级迭代，以及 AI 协同开发与交付流程。全部代码以细粒度 commit 提交，开发对话以 Prompt 快照形式随代码入库，可逐提交还原协同过程。

按比赛要求的三个重点，本文档把两块内容独立成章：第 6 章「意图控制策略」说明系统如何在模型之外仍然握有「这句话算不算办事、办哪一件、办到什么程度必须停下来问」的决定权；第 11 章「可复现性论证」不满足于「我这台机器能跑」，而是给出干净克隆与本机的逐字对账、以及三处「总账写着 0 失败、实际有红」的实证；架构设计见第 3–5 章。

提交清单对照

比赛要求	本项目落地情况
Public 仓库 + 细粒度提交历史	GitHub Public，1056 个独立 commit，横跨 39 个开发日（2026-09-23 → 11-04），禁止一次性打包
AI 对话历史 Prompt 快照	ai-prompts/ 共 232 个 .md = 189 份 Prompt 快照 + 42 份施工单原件 + 1 份索引，与代码分开提交
README 复现指南	仓库根 README.md（环境 / Key 两种配法 / 启动 / 自检 / 演示话术）
5-8 分钟演示视频（含 AI 协同）	docs/演示视频分镜.md（逐镜 + 解说词，本地录屏）
技术文档 PDF ≤30 页	本文件
开源协议	MIT License（仓库根 LICENSE）

1. 项目背景与定位

学生在校园里要同时盯课表、待办、取件、各类咨询，信息散落在不同系统。校园时间管家用「一个助手」把这些收口，分三层：

- 学生活端（旗舰）：课表时间规划 + 校园问答 + 快递外卖查询。学生说一句话（如「把游泳排到周四 19:00」），系统算出提案、弹确认卡，点确认才写进日程。
- 驿站 / 商户侧：监听快递外卖台账变动，主动推送取件提醒与滞留预警，用自定义人格智能回复咨询。
- 管理控制台：零代码维护知识库、客服人格，并在线配置大模型本身。

三层共享同一个 Agent 引擎（判定 + 协调执行）+ 知识库 + 大模型，避免重复实现。

2. 技术栈与运行环境

层	选型	说明
后端	Python + FastAPI	把函数变成网页可调用接口；uvicorn 启动
前端	原生 HTML / JS	无需打包工具，不引入前端框架；确认卡走内嵌 /confirm 页面
大模型	DeepSeek API	Key 走环境变量 / .env 或管理后台，不写进代码
Agent	自写轻量 ReAct 循环	不套重框架；确定性分支优先于模型
存储	JSON 文件（app/data）	由 CAMPUSTIME_DATA_DIR 指向，测试用临时副本
测试	FastAPI TestClient + 自写 Checker	无需 pytest，评委零额外安装即可跑

运行（复现摘要）

```
pip install -r requirements.txt
cp .env.example .env # 填入 DEEPSEEK_API_KEY（或启动后在 /admin 在线填，免重启）
uvicorn app.main:app --reload
# 浏览器打开 http://127.0.0.1:8000
python tests/run_all.py # 全量自检：失败 0 项（6039 项 / 111 批，2026-11-04 实测）
```

3. 系统总体架构与工作流

一次「学生说话 → 出提案 → 确认写库」的请求流：

```
学生输入
└─> /api/chat
├─ 【判定层】先分「问 / 办」，再定模块
│ ① 样本库模糊匹配 → ② 语义层问一次 LLM 只要 {module,intent}
│ ③ 关键词表兜底 （前端也可显式指定 module）
├─ 确定性分支(3 / 3a-bis / 3a-c / 3b / 3c / 3c-bis / 3c-pre~pre5 /
│ 3c-ter / 3d / 3d-ret) 命中即出提案，不调模型
│ ├─ is_confirmation+peek_pending 学生回「确认」→ 系统写/删库
│ ├─ wants_clear_timetable 出清空提案+确认卡
│ ├─ wants_add_todo 出待办提案+确认卡
│ ├─ wants_list_todo 纯读，列出待办
│ └─ missing_thing_of 『看不见 X』真查库核实
└─ 都没命中 → 模型(ReAct) 用 read-only 工具产出提案
提案 = options[]，每条带系统盖章的 kind
```

```
└─> 前端 renderOptions 按 kind 渲染确认卡
└─> 学生点确认 → 系统写库（模型全程不参与）
```

关键点：写库的唯一入口是系统侧的确认动作，模型无论走哪条路都碰不到写入接口。这是「学生没点头就宣布已经删了/加了」这一类事故的硬防线。

3.1 一次完整请求的时序（以「帮我把游泳排到周四」为例）

阶段	发生什么	谁在做	写库?
① 判定	先分「问 / 办」，再定模块；同一次判定给出 {module=planner, intent=add_todo}	判定层（样本库 → 语义层 → 兜底）	否
② 算账	读周表，避开已有课程与待办，摊出 2-3 个可行时段	执行层（planner 确定性代码）	否
③ 出卡	写成 options[]，系统盖上 kind，存入待确认暂存柜（TTL 3 小时）	执行层	否
④ 确认	前端按 kind 渲染确认卡；学生点【确认】	前端 + 系统	此时才写
⑤ 回执	从暂存柜取出提案执行，回「已安排」，周表/月表立刻可见	执行层	已写

注意第 ① 步和第 ③ 步是两件独立的事：判定层只负责「这句话想干什么」，它给出的结论会被喂给执行层那套老代码，而它自己一步都跨不进执行层。

3.2 模块边界（谁有权做决定）

目录结构本身就是这条边界的证据：判定、执行、存储三件事，分别落在三个互不越界的目录里。

目录 / 文件	规模	有权做什么	无权做什么
app/agent/intent.py	754 行	判「问 / 办」、定模块与意图、给 source 与置信度	不生成事名、不读业务库、不写任何东西
app/agent/readonly.py	83 行	声明哪几个工具可以交给模型（READONLY_SPEC）	不实现业务逻辑，只做白名单
app/agent/pending.py	230 行	提案暂存、取回、过期（TTL 3 小时）	不决定提案内容，也不执行提案
app/agent/engine.py	275 行	跑 ReAct 循环，把工具结果整理成提案	没有写库接口；提案一律交回主流程
app/modules/planner.py	7308 行	查空档、摊候选、抠事名与日子、算时长	不写库、不弹窗；它只算出提案
app/store.py	—	唯一的数据读写层（按学号分目录）	不认识「意图」，只做存取
app/main.py	7347 行	路由、确定性分支、确认后的执行、出口复核	不自己发明排期规则（调 planner）

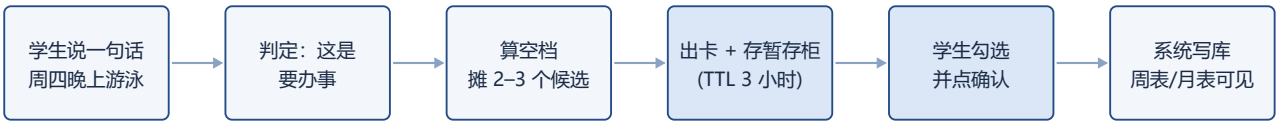
这张表的用处是回答一个具体问题：当一件事办错时，该怪谁？「事名读错了」在 planner；「这句话被当成办事」在 intent；「没弹卡」在 kind 或 main 的分支顺序；「弹了卡但没写库」在 pending 与确认链路。把责任落到文件级别，是这个项目能在一千多笔提交里保持可维护的主因。

3.3 三条端到端业务工作流

上面是「一次请求怎么走」，这里是「一件事从开头到收口怎么走」——两者不是同一张图。前者决定一次交互对不对，后者决定状态会不会丢。三条线各有一个明确的终点和一份状态词表。

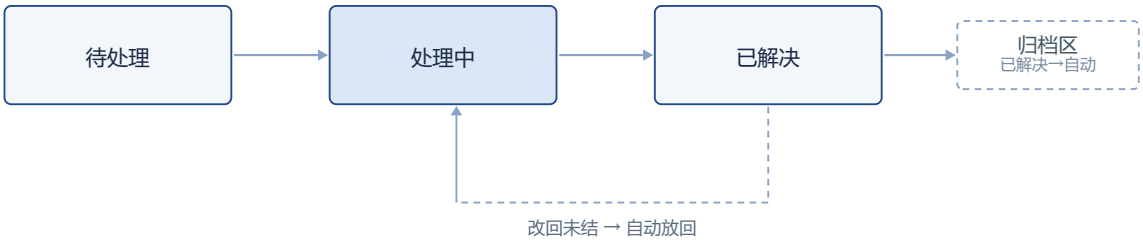
工作流	起点	中间状态	终点（收口条件）	谁保证状态不丢
① 排程闭环	学生说一句要做的事	候选 / 已出卡（暂存柜，TTL 3 小时）/ 已确认	落到周表与月表，可改期 / 删除 / 标记完成	pending 暂存柜 + kind 盖章
② 报修闭环	低代码一句「XX 坏了怎么报」→ 服务定义 → 过白名单 → 落库	学生报修 → 工单待处理 → 处理中 → 已解决（台账里一条记录同理）	「已解决」自动归档（办完就收，不在表里越堆越多）	一份状态词表 + 归档联动
③ 取件闭环	快递 / 外卖台账变动	待取；到件 ≥3 天未取 → 标记滞留并主动提醒	学生取件后不再提醒	台账是唯一事实来源，提醒由状态推导

图 3-1 ①排程闭环（高亮格 = 唯一等你点头的那一步；之前一切都没落库）



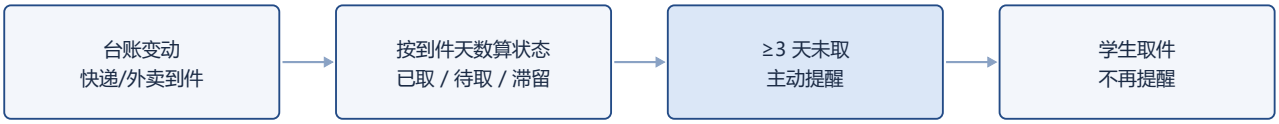
未确认前：库里一个字节都没变（可取消、可过期）

图 3-2 ②报修闭环：三态与归档联动（服务定义由低代码一句话生成，状态与工单共用一份词表）



非法状态一律 400 并列出可选项；工单与服务登记记录共用这一份词表 _WO_STATUSES

图 3-3 ③取件闭环（「滞留」由到件时间算出来，不是人工标记）



把「已提醒」存成状态，就会出现「提醒过了但件已取走」的鬼数据

三条线里最容易被写坏的是状态：只要多一个人手写一份状态字符串，就一定会有两处对不上。所以这里有一条硬约定——状态只有一份词表（main.py 的 _WO_STATUSES：待处理 / 处理中 / 已解决），工单与服务登记记录共用同一份，不新开第二张状态表；非法状态一律回 400 并列出可选项，而不是静默接受。

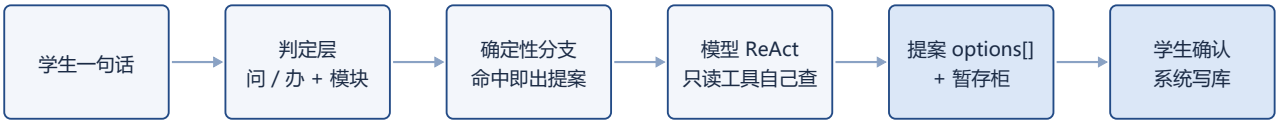
归档这条也有反向要求：「已解决」触发自动归档（办完就收起来，用户拍板过），但从「已解决」改回未结时必须自动放回——只做正向，误标一次就得去归档区里捞人。这类「正向顺手、反向没人做」的坑，是状态机最常见的坏法。

取件那条的判据值得记一笔：「滞留」不是一个人工标记，而是从到件时间算出来的（≥3 天未取）。这样它不可能与事实不一致；反过来，若把「已提醒」也存成一个状态，就会出现「提醒过了但件已经取走」的鬼数据。

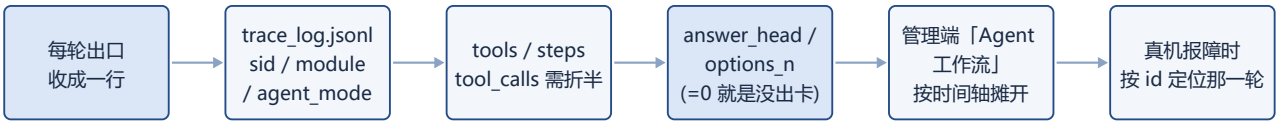
3.4 Agent 工作流：一轮对话到底留下了什么痕

模型参与的那条路（开放问答、需要自己查资料的问题）跑的是自写轻量 ReAct 循环。每一轮结束，系统把它收成 trace_log.jsonl 的一行，管理端「Agent 工作流」视图按时间轴摊开。这不是日志洁癖，而是判断「它是不是真的办了事」的唯一证据。

图 3-4 一轮对话的工作流（上半：请求怎么走；下半：它留下了什么痕）



分支只有一条会走：命中确定性分支就完全不调模型（省一次调用、也更可复现）



tool_calls=2 且 options_n=0 的记录，精确描述了「忙了一场、什么也没交付」

字段	含义	为什么必须有
sid / owner	会话标识与身份	同一时刻多个学生在用，混起来就无法复盘
module / agent_mode	走的是哪个模块、有没有开 Agent 模式	区分「模型答的」与「确定性分支答的」
tools / steps / tool_calls	调了哪些工具、几步、几次	见下面那个折半的坑
answer_head / options_n	回答开头、给了几个可点的候选	options_n=0 就是「没出卡」
trace	思考轨迹（人话版）	学生核对「它有没有听错」的凭据
ts / id	时间戳与行 id	真机报障时能直接定位到那一轮

两个实测出来的坑，都写在这里免得下一个人再踩：

- tool_calls 必须折半：引擎每调一次工具，在 trace 里占两条（「调用工具」+「工具结果」）。直接 len() 得到的是条数不是次数，会让人误判「它查了 4 次」。
- tool_calls 与 options_n 要一起看：真机有一行是 tool_calls=2 而 options_n=0——意思是它确实动手查了，却没给使用者任何可点的东西。这一格比任何报错都准，因为它精确描述了「忙了一场、什么也没交付」这种最难自查的失败。

为什么把这块写进架构章：可观测性不是运维附属品，而是判断「AI 到底做了什么」的前提。没有这一行记录，「它没弹卡」和「它弹了卡但你没看见」在学生和开发者眼里是同一句话，而这两件事的修法完全相反。

4. 核心设计一：Propose / Execute 分离

为什么

早期用纯模型直接回「已经帮你排好啦」，学生刷新日程却空空如也一模型没有写库能力，只是在文字里「表演」写库。把「说要做」和「真做了」混在一起，学生无法区分，是最伤信任的故障。

怎么做

- Agent 只调用 read-only 工具（查空档、列待办、读文件）。任何会改变状态的操作，都先变成一张「提案」返回，绝不在对话轮里直接落库。

- 提案由系统盖章 kind（如 todo_add / todo_remove / todo_mode / todo_slots / timetable_clear）。前端只按 kind 渲染对应确认卡——显示与否由系统决定，不由模型自报。
- 提案先存进待确认暂存柜（app/agent/pending.py：内存 + 磁盘双写、3 小时 TTL）。学生点【确认】后系统才取出来执行；点【取消】或过期则一切不变。

「给 Agent 的提案做标记、前端识别后渲染」的落地形态是 kind，且必须由系统盖，不能由模型盖（否则「显不显示弹窗」的决定权又回到模型手里，正是「经常不弹弹窗」的成因）。

4.1 kind 全表：系统盖的 15 个章，与它们各自能写什么

这张表是「显示权与写入权都在系统手里」的具体形态。每一行对应前端一种卡、以及确认后系统允许执行的一个动作；表外没有第 17
种——模型想出一个新的表达方式，系统也不会因为「它说得很像」就多出一种写库路径。

kind	卡片长什么样	确认后系统写什么
todo_add	单条待办确认卡（事名 / 日子 / 起止）	新增一条待办
todo_add_batch	多条待办确认条（一次点完）	批量新增
todo_slots	候选时段卡（带复选框，可勾多条）	按勾选的段各写一条
todo_remove	单条删除确认卡	删掉那一条
todo_remove_batch	「这 N 条全都要删」批量条（#117）	一次删掉这 N 条
todo_remove_candidates	候选清单（只读，供学生指认第几条）	不写——它只是清单
todo_done / todo_done_batch	标记完成确认（单条 / 批量）	把状态改成完成（重复待办按天记）
todo_done_candidates	标记完成的候选清单	不写
todo_retime_candidates	改期候选（同一件事的多个可选时段）	不写，选中后再出提案
todo_pick	「你说的是哪一条」（#111：同名多条时让学生点选）	不写
todo_mode	日程展示模式（周表 / 月表）	只改视图偏好
timetable_change	课表变更确认（导入解析结果先预览）	整表替换（必须经预览）
timetable_clear	清空课表确认（高危动作，单列一种卡）	清空
work_order_report	报修上报确认（服务定义决定字段）	生成一条工单

注意三类「只读候选」的存在（todo_remove_candidates / todo_done_candidates / todo_retime_candidates）：它们是「系统把话说清楚」的产物，不是待确认的提案。把它们做成只读，是为了避免一种很常见的坏设计——把「我列出来给你看」也做成一个可以点"确认"的按钮，学生会以为点一下就全删了。

5. 核心设计二：判定层 / 执行层分离

本项目最重要的一条设计，也是它和「随便套个 LLM 问答」的根本区别。

5.1 它要解决的问题

学生在报障时说了一句一针见血的话：

> 「很多的字都是接不住的，你现在能接住的都是我测试给的。」

过去每报一次障就往词表里加一条：planner.py 里堆到 30
多张关键词/正则表，router._QUICK_MAP 还有 47 个手写词（另有 5 个由校历词表并入、合计 52

条，见下），全是 if keyword in text 这种朴素子串匹配。这是在拿字符串匹配干语义理解的活一中文口语的写法是无限的，「你说一句我补一词」永远慢一拍，词表加多长都收敛不了。

52 这个数怎么来的：_QUICK_MAP 的字面量里手写了 47 个键，末尾还有一处 **{w: "faq" for w in CALENDAR_FACT_WORDS}，把校历词表整张并进来——那张表是 5 个词（开学 / 截止 / 报到 / 放假 / 校历），47 + 5 = 52。所以严格说只有 47 个是「手写硬编码」，另 5 个是从别处派生的；09-26 首次统计时记为「50 多个」，与本版不矛盾（那时校历词表还没并进来）。

5.2 解法：把「听懂」和「执行」切开

图 5-1 两层边界（上下两侧的权限差别，就是这条设计的全部内容）



模型最多影响上面那一层的候选排序；下面那四件事，它一步都跨不进来。

5.3 为什么这条分离没有破坏 Propose / Execute 的地基

语义层那个大模型干的活只有一件：把这句话听懂，输出结构化意图。它拿不到任何写库接口，也碰不到排期算法。拿到 JSON 之后，查空档、摊候选、出卡、写库走的还是原来那套确定性代码。所以「AI 只提案、系统才执行」一条不破一变的只是「听懂人话」这一步不再由正则独占。

5.4 2026-09-27：顺序倒成「语义优先、关键词兜底」

档	做什么	成本	没配 Key 时
① 样本库	历史成功原话 Dice 模糊匹配（阈值 0.42，词长 <4 不匹配）	零	照样能用
② 语义层	问一次 LLM，只要 {module, intent}，置信度 <0.6 丢弃	一次调用	自动跳过
③ 规则兜底	词表命中就走	零	兜底

此前是「关键词第一、语义兜底」，结果是判定权仍握在词表手里，学生一换说法就掉回模型。倒过来之后，类型（module）与意图（intent）由同一次判定一起给出，_QUICK_MAP 从「第一判据」降为「兜底」。

5.5 为什么语义层可以反过来复核规则

规则判了「要办事」，可这句话其实是个问句，或者人就在问答型栏目里（校园问答 / 快递 / 外卖，不含 planner）——这时让语义层复核一次。但否决闸门只在两种正面肯定下才生效（这句带提问信号，或语义层明确说 chat），不是「结论不同就否决」：样本库是模糊匹配，误否决一张本来就该出的卡，比漏判更让学生恼火。多义词（「安排」既是名词又是动词）是关键词表的天生盲区，补词补不好，用这招才治得住。

取向：宁可漏判掉回老路，不可替学生办错事。白名单之外不收、置信度不够不要、没配 Key 就安静返回 None。

6. 意图控制策略

本章回答一个问题：当「听懂人话」这一步被交出去了（交给模型），系统凭什么还算是受控的?

6.1 意图控制不是提示词工程

提示词约束的是模型的行为（措辞、格式、语气）；意图控制约束的是系统的行为边界——一句话能不能触发写库、出卡之前必须问什么、模型说得再像也不能越过哪条线。两者的失效模式完全不同：提示词失效，模型说错话；意图控制失效，学生的事被办错、或者被「表演着办好了」。本项目所有关于「AI 靠不靠得住」的争论，最终都落在后者，而不是前者。

这条边界由三件东西组成，且三者不可互相替代：判定层决定「这句话想干什么」、提案层决定「以什么形状交给学生」、执行层决定「什么时候真的动库」。模型最多影响第一层的一个候选排序。

6.2 四层判定链，以及顺序为什么不能换

层	做什么	成本	失手时的样子	没配 Key 时
① 样本库	历史成功原话 Dice 模糊匹配（阈值 0.42，词长 <4 不匹配）	零	借错字段，说出学生没说的内容	照常可用
② 语义层	问一次 LLM，只要 {module, intent}，置信度 <0.6 丢弃	一次调用	给出高置信度的错分类	自动跳过
③ 规则兜底	30 多张词表 + _QUICK_MAP，命中即走	零	硬判一个错的分类，或装没听见	兜底
④ 模型 ReAct	拿只读工具自己查，产出提案	多轮	编造事实、表演执行	不可用

顺序的判据不是「谁更准」，而是「谁更便宜、谁更可解释、谁在没有网络时还活着」。越靠前越便宜、越能被指认来源、越能离线跑；越靠后越聪明、越不可复现。所以设计取向是：能用便宜的就别用贵的，而便宜的必须「宁漏不误」——它只被允许下自己百分百确定的结论，拿不准就往后交。

这条顺序是倒过来过的，代价有实证。初版是「关键词第一、语义兜底」，结果判定权仍握在词表手里：学生报障「很多的字都是接不住的，你现在能接住的都是我测试给的」。倒成「语义优先、关键词兜底」之后，类型（module）与意图（intent）由同一次判定一起给出，_QUICK_MAP 从「第一判据」降为「兜底」（见 8.2）。

那③为什么还能留在②后面？因为规则认得的那批东西是硬信号，不是猜测：「确认」「取消」这类应答词、「看不见 X」这类明确句式、已经点过头的确认卡。硬信号必须被无条件放行，不许模型改动；但它只在①②都没结论时才轮到——次序保证不了准确率，只能保证「先便宜后贵」这条不破。

6.3 白名单：把「能办什么」写成代码，而不是写成嘱咐

- 意图白名单（app/agent/intent.py）：模型只能从白名单里选一个 intent。选不出、选到白名单外、或置信度不够，一律安静返回 None，由兜底层接。
- 只读工具白名单（app/agent/readonly.py 的 READONLY_SPEC）：模型能被交给哪些工具，是一张显式清单。「不给到模型手里」比「在提示词里写不许用」可靠一个量级——后者是嘱咐，前者是没这条线。

白名单有两个作用，第二个常被忽略：一是收窄，二是可审计。因为白名单之外的路径不存在，运维只需要审计白名单内的组合；而 /classify 这个只读诊断口会摊开「这句是谁判的、判成了什么、置信

度多少」，让「为什么这句被当成了办事情」永远有一个可指认的答案。

6.4 判定层不许说谎：三条规矩

规矩	具体含义	违反后的样子
① 结论必须可指认来源	每一条判定都要带 source（样本库 / 语义层 / 规则 / 模型），/classify 能复现	出错时无法定位是哪一层判歪的，只能整段重写
② 判定层不做内容生成	它只许输出 module / intent / 结构化字段；事名与时间由执行层算，或如实追问	「我想去健身」被写成「游泳」：说 A 回 B，且看不出错在哪
③ 算不出来必须交出去	#76 的让渡口：落在能力缺口上的句子交给引擎查，并记进缺口账（intent_gaps.jsonl）	硬判一个错的分类，比不判更贵——学生会以为系统听懂了

第三条是最反直觉的一条。系统的本能是「总要给个答复」，但在这里，给一个错答复的代价高于说「这句我还排不了」——因为错答复会被学生当成已完成的事实写进他的记忆里，而缺口账至少留了下次补的线索。让渡的前提是知道自己不会什么：没有能力清单的让渡，等于把判断权整个交出去。

6.5 施工单本身就是一种控制手段

本项目从第 70
单开始，所有改动都以「施工单」形式先写下来，再交给施工侧执行。单子的固定骨架是五段——〇 现场事实 / 一 修法 / 二 判据 / 三 不做的事 /
给执行方的三句。这不是文档洁癖，而是把「意图控制」从代码里延伸到了开发过程本身：

段落	控制什么	不写会怎样
〇 现场事实（带行号、带实测）	控制改动依据：先证明现象，再谈修法	症状相同就套上次的修法——同一句报障在本项目报过 4 次，每次根因都不同
一 修法	控制改动位置：改哪几行、为什么是这几行	顺手重构，一次提交牵动三个模块
二 判据（含变异）	控制完成定义：修完必须看到什么，以及判据本身是否有牙	「改完了」变成自证：总账全绿，实际没测到那一格
三 不做的事	控制改动边界：显式列出这次不碰的东西	边界只存在于写单人的脑子里，执行侧无从知晓

其中「不做的事」这一节最容易被当成客套，实际是唯一的负向约束。正向要求（改什么）执行侧一定会做，负向要求（别碰什么）只有写下来才存在。#114 就是一例：单据里明确圈了
candidate_slots / plan_todo_slot / todo_slots_proposal
三支已收，不在本单范围内，执行侧才没有为了「一次修干净」去动它们。

单子也有它自己的失效方式，本项目遇到两种，各配了一条补救：

- 行号会漂：代码一改，单子里写的行号就过期。补救是「已提交快照不改结论，发现不实另立更正单」——#114 开头那节 §0.0
  行号勘误就是：六处要改的仍在原位，只有三处「别动」标记的注释行漂了 50
  行上下，于是另起一节标明漂移，正文一字不改。
- 判据会假绿：一条断言可能因为「今天正好是周三」而永远成立。补救是两条硬规矩——变异必须打在判据上游（打在下游是哑弹：返回码是 0，但那一格根本没测），期望为空的断言必须配一条非空正控（否则程序什么都没做也算通过）。第 11.4 节给了三处实证。

6.6 控制的代价（诚实记账）

控制越强，覆盖面越窄，这是同一件事的两面，不能只报收益：

- 「宁漏不误」的取向让部分口语说法掉回模型反问，学生要多说一句。这是买来的确定性，不是免费的。
- 判定层的语义闸门否决只在两种正面肯定下生效（这句带提问信号，或语义层明确说 chat），不是「结论不同就否决」。误否决一张本来就该出的卡，比漏判更让学生恼火。
- 白名单之外的意图直接不可达：学生真提出一个系统没设计过的需求时，系统的表现是「没听懂」，而不是「试试看」。
- 判据层自己也照不到作用域——静态全绿不等于真机能跑（详见 11.4 与第 13 章的局限一条）。

7. 确定性意图路由（/api/chat）

为减少「模型自由发挥」导致的不可控，能在代码里定死的意图都收归确定性分支，模型只处理真正开放的问答。

分支	触发	系统行为	写库?
确认执行	is_confirmation + 有 pending	按暂存提案写/删库	确认后才写
清空课表	wants_clear_timetable	出清空提案 + 确认卡	否·待确认
加/删/改待办	wants_add_todo / remove / retime	算时间 + 出确认卡	否·待确认
读待办	wants_list_todo	list_todos() 真查并念出	否（纯读）
看不见 X	missing_thing_of	真查库核实，在/不在如实说	否

代码里的分支标记与行号（评委可以逐条对到 app/main.py；标记是注释里的原样写法，没有「3a」这个裸编号——它只有 3a-bis 与 3a-c 两兄弟）：

标记	行号	管什么
3)	2796	确认执行：学生回「确认」就当场写/删
3a-bis)	3143	候选卡确认：学生点【确认所选】→ 写他选的那个时段
3a-c)	3164	确认落空的兜底：暂存里已没提案，把原话捞回来重算
3b)	3416	确定性清空：明说「课表全删了」
3c)	3462	确定性加待办：给全了「某天某点某段」
3c-bis)	4502	确定性加课：「周一加一节体育」
3c-pre)	3492	学生嫌系统挑的点不合适、自己报了个时刻
3c-pre2) ~ pre5)	3510 / 3548 / 3597 / 3615	候选之后自己报点 / 说「你自己定」/ 单条条上说「我自己挑」/ 语义兜底收尾
3c-ter)	3996	「看不见 X」的报障：不是要加新事，是问在不在
3d)	3748	读已有待办（纯读，不写库、不弹窗）
3d-ret)	3805	改期：已排进日程的待办换个时间

「不弹弹窗」的根因通常是：一句话没被任何确定性分支接住、掉进模型，模型只回纯文本（无 options/kind），确认卡自然不来。因此扩大确定性分支的覆盖面是修复此类问题的主战场；而扩大覆盖面靠的是判定层，不是继续加正则。

8. 关键迭代：六次架构级改动（2026-09-26 → 11-04）

这里挑的是改变「判定层 / 执行层」边界的那六次；其余较大的功能与工程改动（知识库分块检索 #40、流式 #07、时区收口 #56、可观测性 #64）在第 9 / 10 节与 ai-prompts/ 索引里逐单可查。

8.1 第一次：引入判定层（09-26）

问题：学生反馈「很多的字都是接不住的，你现在能接住的都是我测试给的」。研判确认 planner.py 已堆 30 多张词表、router 里 50 多个硬编码词（09-26
当时的统计），属于「用字符串匹配干语义理解的活」，加词无法收敛。

文件	改动
app/agent/intent.py	新增：意图白名单 + 问句识别 + 样本库（Dice 模糊匹配）+ 语义层（LLM 只读意图）
app/agent/router.py	关键词落空时先查样本库，再问语义层
app/main.py	/api/chat 接入语义兜底，规则接不住时不再直接掉模型
tests/test_08_intent_fallback.py	新增第八批护栏，覆盖样本匹配、语义闸门、降级路径

8.2 第二次：类型判定整体重做（09-27）

起因：学生截图反馈「校园问答」栏问「我要去哪里取快递」竟出了待办候选卡。第一次修复只堵了那一句话，学生随即指出「你只是修了这一个，其他类型的区分没改」—问题不在那一句，而在「归哪个模块」的判定方式本身。

研判：「归哪个模块」由 router._QUICK_MAP 的子串匹配说了算（当时是手写 47 个键、运行时 52 条），「想干什么」另由 30
多张表负责，两处互不通气。于是出现「模块是校园问答、意图却是加待办」这种自相矛盾的判定。

改动：把类型也收进判定层，与意图做同一次判定，顺序倒成「语义优先、关键词兜底」；/api/chat 的判定层前移到最前，五条确定性分支共用同一次判定；否决闸门全分支让路；新增 QUESTION_RE / looks_like_question / _QA_MODULES—先分「问 / 办」，粗信号只决定要不要复核、不下结论。

两次自踩自修（值得记录）：

- 语义层把口语翻成规范话喂老解析器时，规范话把原话里的信息丢了（「帮我安排周二的游泳」里的「周二」不在语义字段中 →
  候选摊满一整周）。修正：规则认得的句子要用原话解析，规范话只在规则读不出来时兜。
- 模块顶部 from X import Y 是导入时绑定的：Router 在 __init__
  里建死客户端后，打桩/换配置都传不进来，语义层会安静地退回关键词表（异常被 except 吞掉）。修正：按需在函数内 import 才看得到补丁。

8.3 第三次：错别字纠正与思考过程透明化（09-30）

起因：学生报障截图一句话一「没有展示思考过程，没有修改错别字」。他打的是「中文12点的车」（「中文」是「中午」的笔误），管家回的卡片写着「中文的车」，错别字一字未改，而且「12点」也整段丢了。

研判：这句话只有两条，但每条都是一整类问题。查下去发现三个独立根因，其中第一个与「错别字」无关，却比错别字严重得多：

层	问题	后果
样本库	模糊匹配命中就把整条字段抄走	「我想去健身」撞上「…游泳…」(0.73) → 卡片写「游泳」；「练会儿钢琴」撞「…吉他」(0.82) → 「练吉他」。学生说 A，系统回 B，且看不出错在哪

层	问题	后果
规范话	to_canonical 只拼 start and end	学生只说一个起点（「12点的车」）→ 整个钟点被悄悄丢掉 → 规范话退回原话，模型明明读懂了也白读
规则层	_normalize_clock 只认「读音对、写法对」的口语	「中文」不是时间词、擦标题擦不掉 → 整串留在标题里

改动：

- 样本库命中分两档（_SIM_FULL = 0.9）：几乎一字不差才算「全命中」、可复用字段；0.42~0.9 只是「像」，只借 module/intent、字段一律清空，要事名的意图继续问模型现读。模糊匹配只配定路线，不配定内容。
- to_canonical 补 elif start：单独的起点也拼进规范话（end 缺失交给 planner 按默认时长补）。
- 规则层新增 _fix_typo_time_words（时间词的形近笔误纠正），接在 _normalize_clock 开头，一处生效两条路（抠标题 + 抠时段）。三道闸门缺一不可：
- ① 位置上是时间词的先原样收下 — 不然会连环改：「今田晚上有课」→「今夜里上午课」（今天 / 今晚 / 今夜互相都只差一个字）。
- ② 长得像时间词的正经词不许动（_NOT_TIME_WORDS + 星期几）— 「上班」和「上午」只差一字，不拦就会把「加个上班打卡」纠成「上午」再擦掉、变成「打卡」，悄悄少一个词，比报错难发现得多。
- ③ 必须处在时间语境（后面紧跟钟点或星期）才纠 — 取向是宁可漏纠：没钟点的笔误（「明天交电费」）交给语义层，这条兜底只为「离线演示也别把错字写进日程」。

思考过程怎么露出来：/api/chat 改成薄壳，主体移入 _chat_impl(req, meta)，在出口统一插一条「我把这句听成 办事 / 问事 | 依据：哪一档 | 事名读成了什么」。为什么必须单出口 — _chat_impl 有一千多行、几十个 return，挨个 return 去加必然漏（实测就漏了「追问」那条，而那正是学生说错话时最该看到解释的）。meta 走参数而不用全局量：同一时刻可能有多个请求在跑，全局量会把 A 学生的判定写进 B 学生的回复。

学生端把轨迹默认摊开（可在设置里关）—这是学生核对「它有没有听错」的唯一凭据，并在渲染时过滤三处噪声：空的 ReAct 流程标记、与正文重复的「最终回答」、超长的 answer 截断。顺带把 /classify 的 source_note 与思考轨迹统一到同一张「人话表」（原来各写一张，必然有一处会漂）。

补漏：只报了钟点、没说哪天。改完、1398 项全绿之后，拿学生那句原话在真机上再走一遍，发现只修对了标题、时间还是丢的一回的是「挑方便的勾上」一屏候选时段，而里面根本没有他说的 12 点。根因是 parse_add_todo 的契约「日期和起始时刻缺一个就返回 None」，可「只报了钟点」不等于「没说清时间」。最讽刺的是同一个函数下面几行早就写着该怎么做：「学生自己报了准点 → 就着他给的这个点出条，一条就够」，只是这一档一直没人接。修法：把原来只给「自填时间」用的 span_only_proposal 对外，语义层加待办分支在解析失败时再试它一次；只在语义层这一路兜，规则层故意不动（那里的钟点未必是这件事的时间）。这也印证了一条验收习惯：「全绿」不等于「做对了」，一定要用报障原话在真机再走一遍。

8.4 第四次：规则层学会「交出去」（10-28，#76 / #77）

起因：规则层的词表越堆越厚，但有一类句子它永远堆不完——确定性分支算不出来的说法，它要么硬判出一个错的分类，要么装没听见直接掉进模型。

#76 让渡口：先盘清「系统会什么」（能力清单），落在缺口上的句子才交给引擎去查，不再硬判；顺手把不会的那些记进缺口账（intent_gaps.jsonl）。关键取向：让渡的前提是知道自己不会什么—

—没有能力清单的让渡，等于把判断权整个交出去。

#77 改期接上文：学生只说「换个时间」时不报事名。修法是从上文最近且唯一提到的那条待办接住名字；名字抠不出来就不把全部待办摊开——列二十条让他自己挑，是把选择成本丢回给他。

8.5 第五次：改期从「说假话」到「能办事」（10-28 ~ 10-30，#80 / #81 / #82 / #84 / #86 / #87）

这一族是同一个问题的六个截面：学生说要改期，系统要么编一句「已经帮你改好了」，要么沉默，要么真去改却改坏。六张单按「先说实话、再办成事」的顺序推：

单	症状	修法
#80	没出确认卡却让学生「去勾选」	提示词拆条 + 出口兜底；判据挂在模型原文上，不挂在最终布尔值上
#81	新值等于旧值的「空改动」也出卡	改判「结果三元组是否相同」，并进已有的 unclear 分支，不另造状态
#82	「周末」说了等于没说	接 _WEEKEND_WORDS 生成候选日；不动 _pick_date（它有 34 处调用方）
#84	改不动就沉默	empty 出口带 pool_n，把「我没认出来」和「库里真没有」分开说
#86	引擎手上没有能改待办的手	新增 propose_todo_change(op=retime)；auto_slot 默认 False
#87	同名两条没有出口 / 过去的钟点没有闸	todo_date 只收窄不覆盖；工具层拦 past_slot

这一节的价值不在「修了六个 bug」，在于它把「说假话 → 说真话 → 能办事」这段推进留在了记录里：#80 / #81 / #82 / #84 全在把「系统以为自己办到了」改成「如实说办没办到」，#86 / #87 才是真去办。顺序反了，多出来的能力只会让假话说得更像。

8.6 第六次：候选与批量——系统自己没把话说全（11-01 ~ 11-04，#114 / #115 / #117 / #118）

这一族四单有一个共同点，正好是第 6 章的反面：没有一单是「模型不听话」，全部是系统在把话交给学生之前，自己漏了东西——漏字段、漏日子、漏时长、把一次操作拆成两次点头。

单	学生看到的样子	根因（系统侧）
#115	多候选卡上的星期和时长是空的，勾了也写不进去	模型拼出来的 JSON 少了 kind / weekday / minutes，读取侧只做透传、没做兜底；前端按 kind 渲染，缺 kind 就渲染不出确认条
#117	「一次说删两条」挂出两张卡，要点头两次	「挑中多条」在 planner 里被降级成 many（含糊不清），批量卡早就存在、只是没有入口
#114	说「一次三小时」，排出来是 125 分钟，回答里还写着「按你说的 3 小时」	擦频次时把范围词一起擦了；装不下时不说装不下，改用原话里的时长当结论
#118	没配 Key 时说「周末打篮球，四个小时」，候选落在周三且每段 60 分钟	离线 Mock 那条路写死「今天 +0/1/2 天、准入 60 分钟」——「周末」和「四小时」两个词从没进过它的视野

四单的修法方向也一致：不是加强提示词，而是把系统自己该有权决定的东西收回来——#115 在读取侧兜底补齐字段（模型少给的，系统按已算出的值补）；#117 让「挑中多条」有独立状态，一次确认办完一批；#114

装不下时如实说装不下，并把范围词留住；#118 让 Mock
复用与在线路同一套时长解析（wanted_minutes）与同一把周末窗尺子。

最后一条值得单说：修 #118 时，本可以直接在 Mock
里再写一遍时长正则，但那样就有两份「四小时 = 240 分钟」的实现，两边迟早漂。实际做法是调已有函数，并顺手把事名抠取也一起改——因为确认卡回传的是「确认 日期 起-止 事名」，事名若带着「周末」「四个小时」，这个脏名字会一路写进日程库。这类「同一份判断只许有一处实现」的收敛，比修单个 bug 值钱。

8.7 验证

- 新增/扩充回归：第八批覆盖类型判定重做（问句识别、语义压过关键词、有/无 Key 双路、端到端不出卡）。
- 把学生截图里那两句原话（「我要去哪里取凯迪」「我要去哪里取快递」）钉成永久回归，防止复发。
- 全量 tests/run_all.py：通过 6039 项，失败 0 项（111 批），2026-11-04 实测。这个数字只在自己机器上成立不算数，克隆侧对账见第 11 章。

9. 测试策略

测试不依赖 pytest，只用项目虚拟环境已有的 FastAPI TestClient + 自写 Checker，评委零额外安装即可跑。

批次	覆盖内容
test_01 基础	健康检查、页面路由、知识库 CRUD、客服人格、通知、没配密钥时不裸 500
test_02 上传	上传与边界防护、掩码回显与「掩码串不被写回」防呆、docx/xlsx/pptx 正文与表格解析
test_03 规划	周表导入与校验、空档避课、待办防撞课、月表聚合、会话历史上限
test_04 人设	三套性格接口、规划 Mock「提议→确认→写入」闭环、性格腔调真注入
test_05 学生视图	只读接口数据契约、主面板四视图标记、既有接口回归护栏
test_06 加东西	提案工具直调、日程面板提案、沙箱隔离
test_07 待办意图	三类「代办」口语走系统分支、空列表分支
test_08 意图兜底	样本库匹配、语义闸门、**类型判定整体重做**、残卡字样永久回归
test_09 增删查改	点名删/盲删、改期换时间、语义层直出卡、确认后落库与旧条移除
test_10 标记完成	标记完成改由系统接管（撤掉模型直调写工具）、三分支 ok/many/empty、问答栏不再静默失败
test_11 待办定位	「第N条 / 裸数字 / 连写23 → [2,3] / 最后一条」、有清单时永不返回 empty、撞时间批量删
test_12 批量删	「都删了」指代上一句清单、批量确认条（一张条一次点头）、「每个删一个」去重（与全删相反）
test_13 猜测与证据	上下文四级台阶、只认结构性事实（精确同名 / 候选业务等价）、不做相似度猜测
test_14 语义强弱	语义层单独猜的 done 是弱信号（让给删待办）、规则层硬信号照旧；打桩必须打到 app.main 模块名
test_15 口语样例	20 句真人口语自主探测（软删/错字/语气词/指代/抱怨/知识库错字）+ 沙箱假绿的暴露

批次	覆盖内容
test_16 错别字与轨迹	时间词形近笔误的三道闸门、样本库两档命中、规范话不丢起点、思考轨迹必在（含追问/问答两分支）、只报钟点没说哪天也出条
test_intent_classify	判定层诊断口 /classify 62 项：问/办、模块与意图同判、有/无 Key 双路、只读红线

- 每个测试文件在独立子进程 + 临时数据副本（sandbox）中运行，绝不污染真实演示数据。
- CI 式总账：run_all.py 汇总每批「通过 X 项，失败 Y 项」，有失败退出码为 1。
- 血泪教训已固化为自检手法：沙箱只套一半等于没套。曾有一个测试上半段露在沙箱外，每跑一次全量就往真实暂存文件写一张确认卡，界面上凭空多出待办。现在跑测试前后给几个运行时文件算 md5 对比，必须一字不变。

```
python tests/run_all.py
# 总账：失败 0 项（6039 项 / 111 批，2026-11-04 实测）
```

9.1 11-02 ~ 11-04 新增的护栏批次（11 个新文件 / 290 项；含老文件扩充，总增量 349 项）

这段时间的改动集中在四处：低代码服务（清单进 prompt、触发词撞车剔除、可编辑）、候选卡、批量确认、无 Key 那条路。护栏按这四处落，每一批都带变异（把修法摘掉，判据必须变红）：

批	项数	钉住的是什么
#105 已有清单进 prompt	13	拼进 prompt 的形状：每项最多 5 条、总共最多 12 项、表空回空串——prompt 是要花钱的
#105 触发词撞车剔除	16	撞了内置项或自建项就剔掉、并在回话里点名撞了谁；A 形表 8 / B 话术 4 / C 归属 4
#106 服务定义编辑	20	编辑保 id、两道 exclude_id 闸（改自己不算撞自己）、内置项 400 / 管理端 403
#106 编辑入口（前端）	12	自建服务卡上有编辑按钮（data-svc-edit），内置项在同一个 if 里被排除
#110 相对日期删课	16	「明天 / 后天 / 明早」在删课这一支必须与绝对日期同解
#111 改期同名多条	57	定位的两把尺（第几条 / 哪一天）、候选点选形状、记账读回
#113 第 N 节	16	「第 N 节」= 那天第 N 门课；越界不兜底；不牵动排待办那一支
#114 周末计划	35	A 擦频次仍留范围词 / B 说真话 / C 日子退让 / D 档尾 / S 静态
#115 候选卡字段	30	补字段、收敛真的发生、驱动真引擎、四条红线
#117 批量确认	58	删与标记完成两族对称、三条真机链路、接线成对、前端未动
#118 无 Key 那条路	17	T 批抠名 7 + W 批周末窗 / 准入线 / 诚实话 10

这张表只统计新增文件，所以对不上总账是正常的：批数能对上——11-01 之后仓库里新增了 12 个 tests/test_*.py，去掉属 #98 的 test_99，剩下的 11 个就是表里这 11 行；项数只有 290，而 11-01 收口是 5690、现在是 6039，差 59 项。这 59 项落在既有批次里——同一批改动顺手给老文件补的断言与改口，不产生新文件，因此不进这张表。所以总增量是 349 项 = 290（新文件）+ 59（老文件扩充）。直接拿 290 去对 6039 会差 59，那不是错账，是统计口径写在标题里。

这些批里有两处值得单说，因为它们把「护栏该防谁」这件事说清了：#114 的 S 批是静态检查（不许再出现硬编码的星期偏移），#117 有一条断言是「前端未动」——它防的不是功能错，而是「为了修后端顺手改了前端」这种失控。护栏不只测行为，也测边界。

10. AI 协同开发与交付流程

本项目全程在人与 AI 协同下完成。为满足「细粒度提交 + Prompt 快照」要求，约定：

- 每做一步逻辑改动就单独 commit，绝不在最后一次性打包（本文档版本日为 1056 个，此后持续增加）。
- 每次开发会话写一份 Prompt 快照（ai-prompts/YYYY-MM-DD-<主题>.md），记录「用户 Prompt → AI 研判 → 改动 → 验证」四段。
- 快照文件与代码分开提交（单独 commit），评审把任意快照与其 commit 对照即可还原那一步协同过程。
- 当前 189 份快照（另有 42 份施工单原件/节选，ai-prompts/ 合计 232 个 .md），索引见 ai-prompts/README.md。

10.1 交付链路本身也被记录

仓库推送到 GitHub 的过程同样留了快照（ai-prompts/2026-09-27-github-push-and-contributor-identity.md）：三重独立卡点（代理拦截 → 仓库未创建 → 凭据缺失）的定位与解法，以及「提交归属到底由什么决定」的取证过程—commit 里只有 name 和 email 两行自由文本，GitHub 只按 email 反查账号、完全忽略 name。这类基础设施环节的踩坑与推演，同样属于「AI 协同过程」的一部分。

10.2 协同模式的两个特征

- 先取证再动手：每次改动前先读代码、跑测试、考证具体是哪一行在起作用，避免「症状相同就套上次的修法」（同一句报障在本项目报过 4 次，每次根因都不同）。
- 架构级判断与补丁级修复分开：当学生指出「你只是修了这一个」时，不是再补一个正则，而是回到判定方式本身重做（见第 8 节）。

10.3 一次协同实录：从「我说」到「它反问」到「我拍板」（2026-11-04）

这一节不做摘要，直接把一次完整的协同过程摊开——因为它比任何描述都更能说明人在这个协同里到底做了什么。原始对话的 Prompt 快照见 ai-prompts/2026-11-04-ticket118-mock-weekend-duration.md。

阶段	谁	发生了什么
① 派活	人	只给现象，不给行号：「同一句话『周末打篮球，四个小时』，配了 Key 正常，没配 Key 就落到周三」。并主动说明这条没钉到具体行
② 反问	施工侧 AI	连问五个范围问题：「无 Key」是完全没配还是降级？A/B 是同进程还是两进程？看到的是候选跑偏还是话术里写了『工作日』？「四个小时」要不要一起收？演示前修还是演示后修？
③ 补现场	施工侧 AI	自己去读代码，把现场钉成 main.py:2123 → planner.py:1749，§5 在 :1799-1819，find_free_slots(d, 60) 写死，并指出「今天周三，所以候选正是周三/周四/周五」
④ 拍板	人	① 范围只动 Mock，但必须连 _guess_title 一起改（否则库名会脏）；② 时长复用现成的 wanted_minutes()，不许另写一份；③ 准入线写 ≥，不许写 ==
⑤ 推翻一条	人	施工侧断言「你真机周末有 4 小时空档，正路能出卡」——实测 find_free_slots('2026-11-07', 240) == []，那句话不成立，判据得改
⑥ 施工	施工侧 AI	三笔提交：fix（两处 + 诚实话）/ test（17 项 + 两条变异）/ docs（快照 + 单子入库）

阶段	谁	发生了什么
⑦ 复验	人	真机复跑：出卡 2026-11-07 周六 07:00-15:40（520 分钟），事名打篮球——通过
⑧ 发现假绿	人	复验时顺手打「周五打篮球 / 周六打篮球 / 周日打篮球」，三句的候选全落在今天：那张单子的对照句写的是「周三下午」，而今天正好周三，所以它绿得毫无意义。立即另记一单，不混进已收的这一单

这八步里，人做了三件 AI 不能代替的事：定范围（做什么、不做什么）、推翻一条不实的现场断言、以及在「都绿了」之后仍然去点那几个没被覆盖的句子。AI
做的是把现场钉到行号、按拍板施工、并把判据写成能红的测试。

第 ⑧ 步值得单独记一笔：它说明「测试全绿」和「这件事被验证了」是两件事。那条对照句之所以永远绿，不是因为它写得差，而是因为它挑了一个和「默认起点」重合的日子——今天周三，系统默认从今天起铺，于是「铺周三」这个断言恒真。这类假绿只能靠人在真机上换几个说法去撞，撞出来之后再补一条不与默认重合的对照句。

11. 可复现性论证

本章要论证的不是「我这台机器能跑」，而是别人跑出来的结果，与我跑出来的结果，是不是同一个东西。一个只能在自己机器上复现的系统，本质上无法被评审，也无法被接手。

11.1 三个可复现对象

对象	怎么复现	复现失败的样子
代码	1056 个细粒度 commit，横跨 39 个开发日；按 commit 顺序读，即可看到每一次判断被推翻与重建	一次性大提交：只能看到最终形态，看不到取舍
过程	189 份 Prompt 快照 + 42 份施工单原件，与代码分开提交；任取一份都能与它对应的 commit 对照	只有结果没有过程：无法判断哪些结论是推出来的、哪些是猜中的
结果	tests/run_all.py 总账：通过 / 失败 / 批次数三项；干净克隆侧复跑后逐字比对	「我这儿全绿」——但绿的是别人的环境

11.2 方法：干净克隆，逐字对账

做法是：在另一个目录 git clone 到最新提交，不带 .env、不带任何 app/data 里的历史数据，按 README 装依赖、起服务，两边跑同一批测试，然后比对三项数字。

对账项	结果（2026-11-04 实测，对应提交 ff6e22a）
测试总账（同一批）	严格克隆（tracked-only worktree）6035/0（111 批），本机 6039/0（111 批）。差值 4 项全部落在长期差项 test_86「C 批（数据）」6□2 —— 运行时样本库不入库，克隆侧天然少几条，与本批改动无关；其余逐项相同
管理端 4 个口	/api/admin/kb、/api/admin/messages、/api/workorders、/api/admin/trace 全 200；错串或不带口令 401
学生端 4 个口	/、/student、/api/todos、/api/notices 全 200（学生侧不需要口令）
无 .env 能否启动	能。但每句对话都会降级成「负责回答的服务还没接通」——要看 AI 真回答，必须自己配 Key

第一条是本项目「可复现性」最硬的证据：连 app/data/student/
目录都不存在的克隆，跑出来的失败数与开发机同为 0。这条能成立，靠的是一个刻意的设计——依赖起始状态的批次必须在沙箱里自种一份已知数据（seed_todos / wipe_todos / write_timetable），而不是依赖「开发机上恰好有几条待办」。

版本提示：本文档早先有一版记的是「干净克隆 3102/0、本机 3102/0」，那是 2026-11-01 那次的子集口径（克隆侧跳过依赖可选浏览器的那一批，且当时全量还没到 3000 项以上），与本版不是同一个统计范围。此处统一以 11-04 的全量读数为准，全篇其它地方引用的「11.2 节的 6039 项 / 111 批」指的也是本表第一行。

顺带说明为什么「能起服务」不作为判据：门是 fail-closed 的，没配口令 ≠ 放行。真正的验收是正反两面——对口令 200、错口令 401。只测前者，等于只测了「页面打得开」。

11.3 为什么必须在克隆侧复跑

本机全绿是自证，而自证不可信，因为它同时依赖四样跑完就变的东西：虚拟环境里装了什么、缓存里有什么、app/data 里被历史操作写成什么样、以及那些「等一下再说」的临时代办。克隆侧把这四样全部清零，只留下仓库里真实存在的东西。

克隆侧复跑在本项目里实际抓到过三类问题，都是本机跑一万次也看不见的：缺 .env 时的话术（降级回的句子与文档写的不一致）、数据目录为空时的空态（「周表还是空的」被当成业务结论返回）、口令自举（无 .env 时启动横幅打出的随机口令，与会话中有效期只有一次）。这三类问题的共同点：它们只在「什么都没有」的环境里才出现，而开发者永远不在那个环境里。

11.4 三处「总账写着 0 失败，实际有红」

这是本章最想留下的部分——测试通过不等于被验证。三例都是本项目真实发生过的，而且三种失效是不同性质的。

失效	当时发生了什么	根因
① 沙箱只套一半	有一个测试上半段露在沙箱外，每跑一次全量就往真实暂存写一张确认卡，界面上凭空多出待办	测试自己在污染被测系统；总账仍显示全绿，因为断言看的是它自己写进去的那张卡
② 判据绑在日历上	改期批里一条正控：种一条 3 天后的游泳，断言「挪到周五」落在周五。今天恰好是周二时，3 天后正是周五，「挪动」变成空改动，被「新值等于旧值就追问」的闸拦下，判据必红	只锁了时分，没锁日期（pin_now(9,0)）；判据的真实前提是「今天不是某个特定周几」
③ 结论绑在当前数据上	同一句「真机周末排不下 4 小时」的结论，凌晨与中午两次实测结论相反：凌晨周六只剩 50 分钟空档，中午却有 520 分钟	演示数据被清过；把「当前数据的长相」当成了「系统的性质」

三例指向同一条：失败的方式往往不是「断言错了」，而是「断言测的不是想证的那一格」。①里断言测的是自己刚写进去的数据；②里断言测的是那一天是不是周二；③里断言测的是别人库里有几条待办。这三种都能让总账长期绿着，而系统该有的性质一条也没被证明。

11.5 三条自检规矩

规矩	怎么用	防的是上面哪一例
变异必须打在判据上游	改坏被测逻辑的上游输入，判据必须变红；若只改下游、判据照样绿，说明这条判据是哑弹	①②③通用的「判据有没有牙」
期望为空的断言必须配非空正控	「这里应该什么都没有」必须配一条「同样路径下这里应该有东西」，否则程序什么都不做也算通过	①（测试没跑却通过）
结论必须写明它依赖的数据时点	凡「真机能/真机不能」这类断言，必须挂上实测时刻与当时的占用情况	③（数据漂移）

pytest 那边还有一道门：tests/pytest_gate.py 只做一件事——把 pytest 这个命令接到项目真正的自检入口上，一个子进程跑

run_all，把总账那一行搬进断言消息。它故意不把 40 多份 test_*.py 当 pytest 用例收：那些文件是自带 main 的脚本，其中三份在模块层就 sys.exit，pytest 一 import 就崩，现象是「一行结果都没有」——比测试失败难看得多。这条门在 2026-10-14 之前一直红着（本机 14 条存量失败），根因同样是「断言依赖起始状态」，修法是让那几批在沙箱里自种数据，而不是把演示数据提交进仓库。

11.6 复现命令与预期

```
# 1) 克隆与安装
git clone https://github.com/zhouyuliang0606/campus-time-agent.git
cd campus-time-agent && pip install -r requirements.txt

# 2) 起服务（无 .env 也能起；要看 AI 真回答需配 DEEPSEEK_API_KEY）
uvicorn app.main:app --host 127.0.0.1 --port 8000

# 3) 自检：预期「失败 0 项」，数字与本文档 11.2 节一致
python tests/run_all.py

# 4) 或走 pytest 那道门（等价，但只在 pytest 下可用）
pytest
```

复现若对不上，请按这个顺序怀疑：① 你的 Python 版本（本项目在 3.13 上开发）→ ② 是否漏装依赖 → ③ 是否有批次的输出被截断（逐批看，别只看总账）→ ④ app/data 里是否有上一次运行留下的残留。总账只给三项数字，逐批才是取证的地方。

12. 复现指南（README 摘要）

- 环境：Python 3.10+（本项目在 3.13 上开发与测试），pip install -r requirements.txt。
- 密钥（两种任选）：① 启动后在 /admin 的「API 配置」在线填，立刻生效免重启；② cp .env.example .env 填入 DEEPSEEK_API_KEY。settings.json 已在 .gitignore 排除。
- 没配 Key 也能跑：自动切到确定性 Mock
  模式，排程交互、卡片渲染、知识库问答等离线链路照常演示。
- 启动：uvicorn app.main:app --reload，打开 http://127.0.0.1:8000。
- 自检：python tests/run_all.py（临时数据副本，不碰演示数据），预期「失败 0 项」（对照 11.2 节的 6039 项 / 111 批）。

演示话术（都能直接试）

你说	应该看到
帮我把复习线性代数安排到周四 19:00-20:30	出确认卡；点了确认日程里才出现（没确认不写）
我的代办呢	直接念出真实待办列表（真查库，不是编的）
把周四那个瑜伽挪到周五	出改期卡，确认后旧条目移除、新条目落位
健身显示不出来	系统真去查库核实，如实回答「在的 / 不在」
我要去哪里取快递	走校园问答检索知识库回答，不会被误当成「加个待办」

12.1 没配 Key 时能演示什么、不能演示什么（评委视角）

这一段写清楚边界，是为了避免「照着话术点却对不上」这种最伤的体验。

场景	没配 Key	配了 Key
排程交互（说话 → 候选卡 → 勾选 → 写进日程）	能：走确定性 Mock，卡片、勾选、落库全链路通	能
「周末打篮球，四个小时」这类带范围与时长的话	能（#118 之后）：候选落周六/周日，准入按 ≥ 240 分钟	能
具体某一天的排期「周五打篮球」	不能：离线 Mock 只认「周末」，会把候选摊在今天起三天（见 13 章局限与实测）	能（_pick_date 认周几）
开放问答（校园知识库、快递外卖）	能：知识库检索不依赖大模型	能
需要模型理解的自由表述	不能：会回「负责回答的服务还没接通」	能
课表为空的身份做任何规划	不能：会先回「先去上传课表」——这是设计，不是故障	同左

最后一行是实机验过的坑：离线 Mock 的第一步就检查周表，没有周表时任何规划请求都会得到「先去上传课表」，看不到任何候选卡。所以演示前要先确认登录的那个学号本身带周表。这条不是缺陷——没有周表本来就算不出空档——但它会让「没配 Key 也能看效果」这句话打折扣，必须在文档里说清，而不是等人踩到再解释。

12.2 三个最常见的卡点

现象	原因	怎么办
改了代码，界面上还是旧行为	启动命令没有 --reload（或用的是 run_local.bat）	重启服务。这条在本项目里造成过两次误判，被单独记进 #114 的施工单
管理口全部 401 / 进不去	没配 CAMPUSTIME_ADMIN_TOKEN 时口令是每次启动随机生成的，横幅里那串才有用；抄历史日志里的旧串必然进不去	看本次启动横幅，或在 /admin「API 配置」页按 placeholder 提示填
每句对话都回「负责回答的服务还没接通」	没有 DEEPSEEK_API_KEY（克隆下来默认没有）	在 /admin「API 配置」在线填，立刻生效免重启

13. 局限与后续

- 图片/扫描件读不出字：需要接 OCR；目前只支持 txt / pdf / docx / xlsx / pptx 的文字层。
- 账号体系是演示级：个人库已按学号分目录（app/data/student/<学号>/），但仍然没有密码与鉴权——身份靠登录时选学号，兜底分支不许删。
- 长资料走检索而不是整篇直读：上传文档会切块并建 BM25 索引，问答只带命中的块；整篇直读仍有 6000 字上限，超过就改走 search_docs（不是悄悄截断）。
- 确定性覆盖面仍可扩大：语义闸门偏保守（宁漏不误），个别说法仍会掉回模型反问。
- 未配大模型密钥时的降级路径会说错话：离线 mock 对部分输入给出与事实不符的回答（实测「改期办不成」那句被回成「你周表还是空的」）。演示机已配 Key，正常路径不经过它。
- 改期不携带备注与分类：改期是「删旧 + 加新」，确认后原待办的 note / category 会丢。
- 「加一节 ×× 课」在个别句式上会落到加待办：规则判据 wants_add_course 实测为真，问题出在分支先后顺序（已知缺陷，未在本版修）。
- 加待办工具不收 repeat：propose_todo_tool 无法一次创建「每周/每天」的重复待办（重复待办要用候选卡那条路）。
- 只读工具的查询区间由模型自定：跨周提问时可能只查了一段就答「没有」，与改期工具的全库视野不完全一致。
- 判据层自己也照不到作用域：一层「静态全绿 ≠ 真机能跑」的坑（链 C 实测）。同一套外观规则在静态页上全绿，换到真机的玻璃档下有三处完全没生效——原因不是遍历漏了，而是浏览器里真实的层叠上下文不是静态扫描能算出来的。这类问题只能靠真机截图对照，判据层给不出保证（见 11.4）。
- 离线 Mock 只认「周末」，不认「周五/周六/周日」这类具体星期几：沙箱真机实测（11-04 周三）五句，「周五 / 周六 / 周日打篮球」三句的候选全部落在今天起三天（11-04 周三、11-05 周四），只有「周末打篮球」正确落在 11-07 周六 / 11-08 周日。根因不是「不会算周几」——认周几的判据早就在了：_pick_date('周六打篮球，四个小时') 回 '2026-11-07'，返回的正是那个周六。问题在 mock 的 §5 自己又判了一遍，只查 _WEEKEND_WORDS（('周末','双休')），压根没调用 _pick_date——同一个能力两份实现，离线那份少判了一档。修法是 §5 直接复用 _pick_date（#118 对时长做过同样的收口，日期这一档漏了）。已在 11-04 记明，未修；在线（带 Key）路径不受影响——它走的正是 _pick_date 这条日期解析。
- 演示视频需本地录屏：逐镜脚本见 docs/演示视频分镜.md。

模型仅在开放问答时启用；高可靠操作（写 / 删 /
改）一律系统执行，这条边界不会为了效果而放松。

附录 A：路由与接口清单（84 条 = 77 个 API + 7 个页面路由）

下表逐条列出全部路由（只读接口绝不写库；写库接口都要求携带系统侧的确认动作）。条数由本脚本现读 app/main.py 的路由装饰器得到，与本表行数当场对账——手抄的数会过期，对账不会。

接口	作用
GET /api/health	健康检查
POST /api/chat	主对话入口：判定层 + 确定性分支 + 模型兜底
POST /api/chat/reset	重置某个会话的历史
GET /api/conversation/{sid}	读会话历史（前端刷新后恢复聊天记录）
GET /api/pending/{sid}	读待确认提案—前端据此渲染确认卡，只读不执行
GET /api/timetable	读学生周表
POST /api/timetable/apply	应用课表（整表替换，需先经过预览确认）
POST /api/timetable/clear	清空课表
GET /api/todos	读待办列表
GET /api/todos/month	月表聚合视图
POST /api/todos	新增待办（确认后）
POST /api/todos/batch	批量写入待办
PUT /api/todos/{tid}	修改待办
DELETE /api/todos/{tid}	删除待办（确认后）
POST /api/todos/mode	保存日程展示模式
GET /api/express	快递只读视图（含实时状态标签），供卡片面板用
GET /api/takeout	外卖只读视图，供卡片面板用
GET /api/student-personas	三套学生助手性格预设
POST /api/student-message	学生每轮对话上报（管理员收件箱数据源）
GET /api/notices	学生端读通知
POST /api/upload	上传文件并提取文字
GET /api/uploads	上传文件列表
GET /api/uploads/{fid}	读某个文件的提取文字（AI read_uploaded_file 走这里）
GET /api/uploads/{fid}/download	下载上传原件
GET /api/workorders	读学生工单
POST /api/workorders	提交工单（学生确认上报后才生成）
POST /api/workorders/{wo_id}/status	修改工单状态（管理端）
GET /api/persona	读驿站客服人格
POST /api/persona	写驿站客服人格
GET /api/admin/kb	列出知识库
POST /api/admin/kb	新增知识库条目（零代码维护）

接口	作用
PUT /api/admin/kb/{index}	修改知识库条目
DELETE /api/admin/kb/{index}	删除知识库条目
GET /api/admin/settings	读大模型配置（密钥掩码输出）
POST /api/admin/settings	写大模型配置（掩码串防呆，不覆盖真密钥）
POST /api/admin/settings/test	在线连通自检
POST /api/admin/notify	发布通知（支持附件）
GET /api/admin/messages	学生消息收件箱
POST /classify	判定层诊断口（只读）：摊开 is_question / module / intent / confidence / 是哪一档判的
POST /api/login	学生登录：只认 accounts.json 名单里的学号，没有密码
POST /api/logout	退出登录：只把标签摘掉，不清任何数据（下次登进来课表待办都还在）
GET /api/whoami	当前身份：刷新后仍知道「这是谁的库」，不靠前端那份显示用副本
GET /api/upstream	学生端要显示的那一句话：你的请求会被送到哪台服务器
GET /api/my/workorders	学生查自己报过的工单；快递面板那块「我提交过的工单」的进度就来自这里
GET /api/my/reports	工单 + 自建服务登记合成一张表，附进度（#54）
POST /api/workorders/{wo_id}/archive	归档 / 放回一条工单：台账一个字节不少，只多/摘 archived
POST /api/admin/messages/{mid}/archive	归档 / 放回一条学生消息（软删，要口令）
GET /api/station/workorders	驿站看到的报障列表（倒序，最新在前）
GET /api/station/workorders/archived	驿站「已处理 N 条」点开后的明细（只有读，没有写）
POST /api/station/workorders/{wo_id}/status	驿站处理完改状态——与管理端同一个 store 函数，两个视图不会各说各话
GET /api/station/services	驿站端服务列表：只列归驿站的（后勤的报修不进驿站台账，反之亦然）
POST /api/station/services	一句话 → 服务定义 → 过白名单 → 落库成为可用服务
DELETE /api/station/services/{sid}	删一份服务定义（#39-A：低代码只能一句句长，建错了得能收）
GET /api/station/services/{sid}/records	某服务已登记的记录（服务不存在回 404，不返回空列表装「什么都没发生」）
POST /api/station/services/{sid}/records	登记一条：按服务定义把字段存进台账
POST /api/station/services/{sid}/records/{rid}/status	把一条登记记录标成处理中 / 已解决
DELETE /api/station/services/{sid}/records/{rid}	删一条登记记录（#39-B，真删——台账里那条就没了）
POST /api/custom/records	学生点了确认卡后，系统把这条登记写进自建服务台账
GET /api/admin/service-records	自建服务的登记台账（管理端「后勤报修」那一档的数据源）
POST /api/admin/service-records/{sid}	后勤自己登记一条（电话报修、巡楼看到的坏点，没有学生参与）

接口	作用
POST /api/admin/service-records/{sid}/{rid}/status	后勤改状态：与驿站那个口同一份实现
POST /api/admin/services	管理端建服务：一律归后勤（#54 分流的下半场）
PUT /api/admin/services/{sid}	改一份服务定义。#106：建完就锁死不行——字段、必填、归属都要能改
DELETE /api/admin/services/{sid}	删一份服务定义（#106：建错了得能收；已被引用时回明确原因，不静默删）
PUT /api/station/services/{sid}	驿站端改自己那几份定义（后勤的不许越权改，回 403 而不是装作成功）
DELETE /api/admin/notices/{nid}	撤回一条已发通知（#39-D：发错字、发错人群，以前只能眼看着挂着）
POST /api/admin/kb/probe	试检索探针：管理员自己验「这句话学生问得到答案吗」
POST /api/admin/kb/split	一句话 → 模型按句意拆出的检索关键词（只产建议，不写库）
GET /api/admin/docs	知识文档列表：文件名 / 字数 / 块数 / 索引状态 / 时间
POST /api/admin/docs/{fid}/reindex	按已经存下来的文字重新切块、给老文档补建索引
DELETE /api/admin/docs/{fid}	删一份知识文档，连带它的分块索引
GET /api/admin/usage	用量汇总（管理台那张「用量统计」卡片的数据源）
POST /api/admin/usage/clear	清空用量流水（演示前想从零计数）
GET /api/admin/trace	Agent 工作流列表（只读，不含完整轨迹，点开才取）
GET /api/admin/trace/{row_id}	取某一轮的完整轨迹（只读）
POST /api/admin/trace/clear	清空轨迹流水：只清这本旁路日志，不碰任何业务数据
POST /api/todos/items_add	学生在多选材料卡上勾完、点【加入日程】的落库口（#43）
GET /	登录页：选择身份（学生 / 管理员）
GET /student	学生端主页（四模块）
GET /plan	学生个人日程（周表看课程与待办，月表看哪天有几件事）
GET /settings	学生助手设置页：改助手名字、挑聊天性格、返回登录
GET /confirm	独立的变更确认页面（确认 UI 本身就是一个单独的页面）
GET /admin	管理员端（通知 + 学生消息 + 知识库 + 人格 + 用量 + 轨迹）
GET /station	驿站 / 商户端

附录 B：数据文件

B.1 种子数据（有意入库，管理台可零代码增改删）

文件	内容
app/data/courses.json	示例课表 12 节，产品已不读它：周表实际来自 student/<学号>/timetable.json（store.py:1180），45-A 起 schedule.py 也不读了，现在只有测试拿它当种子（tests/_e2e_env.py 的 demo_courses_from_json，test_05/test_06 与几份 e2e/截图脚本都从它还原周表）；里面的 semester 字段全代码零引用。别按它判断演示机上看到的课表，那份在个人库里。
app/data/kb.json	校园知识库（图书馆/食堂/校车/校医院/宿舍/奖学金…）

文件	内容
app/data/express.json	演示快递台账，8 件，按 owner 分 admin1 / admin2 / demo 三份。其中 P2「运动鞋一双」（admin1，到件 2026-09-19，至今仍「待取」）是故意留的滞留件——express.py:23 定的是 STAY_DAYS_WARN = 3，到件满 3 天没取就算滞留，用它演示「要尽快取，否则被退回」。
app/data/takeout.json	演示外卖订单（制作中/配送中/待取）
app/data/accounts.json	开放的学生账号名单（#38：只认名单、没有密码）

B.2 运行时数据（不入库，已在 .gitignore 设防）

文件	内容	为什么排除
settings.json	管理后台填的模型密钥/地址/模型	含真实密钥
persona.json	驿站客服人格	运行时数据
notices.json	管理员发布的通知	运行时数据
student_messages.json	学生对话上报记录	含隐私
uploads.json + uploads/	上传文件元信息与实体	含隐私
workorders.json	学生工单	含隐私
pending.json	待确认提案暂存（TTL 3 小时）	运行时状态
intent_samples.json	判定层样本库（累积的成功判定样本）	运行时数据
intent_gaps.jsonl	判定层的缺口账（#76：让渡给引擎时记下的原话）	运行时数据
services.json	低代码自建服务的定义（内置种子写在 store.py）	运行时数据
custom/	自建服务的登记台账	含隐私
llm_usage.jsonl	每次大模型调用的用量流水	里面写着会话 id，属隐私
trace_log.jsonl	Agent 工作流轨迹（#64）	含会话 id，比用量账更敏感
student/<学号>/timetable.json	该学生的周表（#32 之后按学号分库）	个人数据
student/<学号>/todos.json	该学生的待办	个人数据
student/<学号>/sessions.json	该学生的会话历史（最多 20 条）	个人数据

数据目录可用环境变量 CAMPUSTIME_DATA_DIR
指向别处——这正是自动化测试的数据沙箱开关：测试把 app/data
复制到临时目录再跑，跑完删掉，真实演示数据一个字节都不动。

附录 C：关键施工单索引（42 份里挑 10 份）

每份单子的完整原件都在 ai-prompts/（文件名 -drop-in.md），与对应三笔提交（fix / test / docs）分开记录。挑这十份的标准是：它们各自代表一种只有过程记录才能说明的判断。

单	日期	它说了什么	为什么值得单列
#76	10-28	规则层学会「交出去」：先盘能力清单，落在缺口上才交给引擎，并记缺口账	「让渡」的前提是知道自己不会什么——没有能力清单的让渡等于全推给模型
#80	10-28	没出确认卡却让学生「去勾选」：提示词拆条 + 出口兜底	判据必须挂在模型原文上，不能挂在最终布尔值上；否则模型说一套、系统判一套

单	日期	它说了什么	为什么值得单列
#82	10-29	「周末」说了等于没说；同一句追问两遍	只加判据、不动 _pick_date（它有 34 处调用方）——收敛改动半径的例子
#84	10-29	改不动就沉默：empty 出口带 pool_n	把「我没认出来」和「库里真没有」分开说，是信任问题不是体验问题
#87	10-30	同名两条没有出口 / 过去的钟点没有闸	todo_date 只收窄不覆盖——一个字段的写权限被明确限死
#111	11-03	改期遇到同名多条，「第一条」「周二的」接不住，问第二遍	死循环类 bug：判据要断在「第二次追问」，不是断在「有没有答」
#114	11-04	说「一次三小时」排出来 125 分钟，回答里还写着「按你说的 3 小时」	装不下就说装不下；单子开头的 §0.0 行号勘误是「不实另立更正」的实例
#115	11-03	多候选卡上星期与时长是空的，勾了也写不进去	模型少给的字段，读取侧必须兜底——不能指望模型把 JSON 给全
#117	11-03	一次说删两条，挂出两张卡，要点头两次	「挑中多条」被降级成 many：状态机少一个态，用户就多一次点击
#118	11-04	没配 Key 时说「周末…四个小时」，候选落在周三且每段 60 分钟	离线路径与在线路径必须共用同一份判断（时长解析、周末窗口、事名抠取）

这十份单子的共同骨架是五段：〇 现场事实（带行号、带实测）→ 一 修法 → 二 判据（含变异）→ 三 不做的事 → 给执行方的三句。第 6.5 节说明为什么这个骨架本身就是一种意图控制手段。

– 文档结束 –